Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Detective Mystery: The Hidden Truth

Team Name: *Team #*	*Truth Seekers* *T152*
Team member 1 (Team Lead)	*Badhani, Karan – 2510360037* *karanbadhani9770@gmail.com* 
Team member 2	*Giri Goswami, Suraj – 2510012585* *2510012585@geu.ac.in* 
Team member 3	*Singal, Khushi – 2510360039* *singalk20@gmail.com* 
Team member 4	*Arora, Tanmay – 2510610178* *2510610178@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Detective Mystery: The Hidden Truth is a detective game written in C++ and set on our own college campus. The sealed Data Structures exam paper goes missing from the Exam Cell in the CSIT basement. The player is the detective who walks between real campus places, searches for clues, reads about the suspects and finally names the thief.

Our main goal has not changed since Phase 1. We want one complete application in which OOP and Data Structures work together so that we can explain why each concept is used for each feature. Since Phase 1 we have made the project smaller and clearer. We kept only the data structures that have a real job in the game: a graph for the campus map, a stack for going back, a hash table for finding clues and sorting for the journal. The suspects, clues and places are built as C++ objects.

In this phase we built the first basic working model. It runs in the console and can be played from the first search to the final accusation. The advanced version with more features and a 2D interface using SFML will be built in Phase 3 (the Final phase).

Updated Project Approach and Architecture

We build the project in stages and each member owns one module. The game logic is kept separate from any drawing code so a 2D interface can be added later without rewriting the game.

System design – The program starts by loading the story from five plain text files (case, locations, connections, suspects and clues). A GameManager class owns all the game objects and runs the command loop. The player types commands such as go, search, back, journal, clue, accuse and the GameManager passes each one to the right object. The result is printed after every action.

Main parts – Location and Map hold the campus as a graph. Clue is an abstract class with three kinds (PhysicalClue, Testimony and DocumentClue) and each kind examines itself in its own way. Suspect holds the details of each person. Case keeps the culprit private. Player holds the current place, the journal and the movement history.

Data structures – The map is a graph stored as an adjacency list. The history is a stack that we wrote as a linked list. ClueTable is a hash table with chaining. The journal is arranged with insertion sort.

Communication, libraries and tools – The parts communicate through ordinary function calls and data comes in only from the text files. There is no network or database. We use only the C++ standard library for now (vector, string and smart pointers). SFML is planned for later. We work in Visual Studio Code with g++, Git and GitHub.

Tasks Completed

Task Completed	Team Member
Wrote the campus mystery (places, suspects, clues and the culprit) and stored it in five text files	All members
Game flow: GameManager, file loading, command loop and accusation rules; Case class with a private culprit	Karan
ClueTable: a hash table with chaining to find a clue by its id	Karan
Location and Map: the campus as a graph (adjacency list) of 7 places, the locked Exam Cell and the go command	Khushi
Player class and our own Stack (linked list), the back command and the journal	Suraj
Clue base class with three kinds, Suspect class and insertion sort for the journal	Tanmay
Unit tests for each data structure and a full playthrough of the case	Each member for their own module
GitHub repository with one branch per member and pull requests(For Phase-3)	All members

Challenges/Roadblocks

Explaining every concept - In Phase 1 we wrote things we could not explain. This time we reduced the list of data structures and made sure each one has a clear job that we can show in the code.

Keeping the project small - A mystery game can grow very large. So we stopped at seven places, three suspects, eight clues and kept the extra features for Phase 3.

Joining four modules. Each of us wrote a different part but the parts needed each other. Player uses Clue and Stack and GameManager uses almost everything so the order of joining them mattered. We agreed on an order for adding the files and took turns on the shared test file so that nobody overwrote another person's tests.

Making the story fair - The story had to be fair to the player. Every clue should lead towards the real culprit, the other suspects should look suspicious without being provable and the right person had to be provable with enough clues. To check this, we played the case a few times going by different routes.

Memory handling in C++ - Writing our own linked-list stack and hash table meant we had to be careful with pointers. We tested an empty stack and colliding keys on purpose and used smart pointers for the clue objects.

Teamwork on GitHub. None of us had worked together on GitHub before. We learned to make small commits from our own accounts and to take turns on files that more than one of us edits.

Tasks Pending

Task Pending	**Team Member (to complete the task)**
Shortest route between places using BFS and our own queue	Khushi
Queue of suspects to question and the interrogate command	Suraj
Keyword search of clues and ranking of suspects	Tanmay
Scoring system and saving and loading progress	Karan
Underground tunnel, main gate, the night guard and two more clues	Khushi and Tanmay
2D interface using SFML on top of the existing game logic	All members
Final testing, report and documentation	All members

Project Outcome/Deliverables

By the end of the project we will have a complete, playable detective game. For this phase we deliver a first working model with:

- one complete mystery with a start, a trail of clues and a solution;
- seven connected campus places, one of them locked until the player finds permission;
- three suspects and eight clues loaded from text files;
- C++ classes for places, clues, suspects, the player and the case;
- the map as a graph, a stack for going back, a hash table for finding clues and a journal sorted by importance;
- an accusation stage that checks the evidence before the verdict;
- unit tests for the data structures and a GitHub repository showing each member's commits.

The final deliverable is the advanced version with more places and clues, shortest-route search, questioning suspects through a queue, scoring and a 2D interface.

Progress Overview

About 60 percent of the full project is done. The core of the game is finished and working: loading the story, the campus map, clues, movement history, clue lookup, the sorted journal and the accusation.

Nothing is behind schedule. The plan for this phase was a basic working model and it is complete. We chose to keep this model simple on purpose so that the advanced version can be shown in the final phase.

The parts still to come are the shortest route, the queue for questioning suspects, searching clues, scoring, more places and clues and the SFML interface. These are planned for Phase 3, the Final phase. We also wrote unit tests for every data structure early so each new feature can be tested against them.

Codebase Information

Repository:
https://github.com/Truth-Seekers09/the_detective_mystery

Branches: In Phase 2 each member committed directly to main from their own account. From Phase 3 we will use one branch per member and pull requests.

Important commits:

- Karan: GameManager with file loading, command loop and accusation; ClueTable hash table
- Khushi: Location class and Map as an adjacency list, locked places
- Suraj: Stack template (linked list), Player class with moveTo and goBack
- Tanmay: Clue base class and three kinds - Suspect class, insertion sort, sorting test

Testing and Validation Status

Test Type	Status	Notes
Unit test: Stack	Pass	Reverse order out; empty pop gives an error
Unit test: Hash table	Pass	All clues found even with every key in one bucket
Unit test: Graph	Pass	Two-way routes; invalid places refused; locked place opens
Unit test: Sorting	Pass	Most important clue first
Unit test: Player	Pass	Back returns places in reverse; empty history gives false
Full playthrough	Pass	Right culprit solves the case; two wrong accusations end the game

Deliverables Progress

- Mystery, campus map (graph), clue and suspect classes: **Completed**
- Back command, clue lookup, sorted journal and accusation: **Completed**
- Unit tests for the data structures: **Completed**
- GitHub repository with every member's commits: **In progress** (continues every phase)
- Shortest route, queue, keyword search and scoring: **Pending**
- 2D SFML interface: **Pending**